PHP 8 & Regex — Guía de Examen

Programación en PHP, POO y Expresiones Regulares | Resumen Completo

REPASO RÁPIDO

1. SINTAXIS BÁSICA Y CARACTERÍSTICAS GENERALES

Fundamentos del Lenguaje

- **PHP:** Interpretado, Open Source, lado del servidor.
- **Sensibilidad:** Case-sensitive en variables (`$x` != `$X`).
- **Terminación:** Cada instrucción acaba en `;` .
- **Delimitadores:** `<?php ... ?>` (recomendado).
- **Comentarios:** `//` o `#` (1 línea), `/* ... */` (multilínea).

Salida de Datos y Cadenas

- `echo` : Imprime varios parámetros. No retorna valor.
- `print()` : Imprime 1 parámetro. Retorna 1.
- `sprintf()` : Formatea cadenas y las devuelve.
- **Comillas dobles (`""`):** Interpretan variables/escape.
- **Comillas simples (`''`):** Texto literal estricto.
- **Concatenación:** Se usa el punto (`.`).

2. VARIABLES, ÁMBITOS Y CONSTANTES

Tipos de Variables

- Empiezan siempre con `$` . No requieren declaración de tipo.
- **Locales:** Visibles sólo en el bloque/módulo.
- **Globales:** Se declaran dentro con `global $var` .
- **Estáticas:** `static $var` preserva valor entre llamadas.
- **Constantes:** `const X = 1;` o `define('X', 1);`

Superglobales (Arrays Predefinidos)

Accesibles desde cualquier punto del script:

`$_GET` `$_POST` `$_REQUEST` `$_SERVER`
`$_FILES` `$_SESSION` `$_COOKIE` `$_ENV`

3. OPERADORES Y ESTRUCTURA DE CONTROL

Categoría	Operadores	Descripción / Nota Examen
Aritméticos	`+` `-` `*` `/` `%` `**`	`**` es potencia (ej. `2**3 = 8`). `%` es módulo.
Comparación	`==` `===` `!=` `!==` `<>`	`===` verifica valor y tipo idéntico.
Lógicos	`&&` `||` `!` `xor`	`xor` exige que solo UNO sea verdadero.
Especiales	`? :` (ternario) | `??` (null)	`$a ?? 'default'` asigna si `$a` es null/no existe.

4. BUCLES Y CONTROL DE FLUJO

Estructuras Iterativas

- `for` / `while` : Condición al inicio.
- `do-while` : Ejecuta **al menos 1 vez**.
- `foreach ($arr as $k => $v)` : Exclusivo de arrays.

Salida de Bucles

- `break` : Aborta el bucle inmediatamente.
- `continue` : Salta a la siguiente iteración.
- `break 2;` : Sale de 2 niveles anidados.

5. ARRAYS Y FUNCIONES INTERNAS

Tipos de Arrays

- **Escalar:** Índice numérico entero (0, 1, 2...).
  `$d = ["Lunes", "Martes"];`
- **Asociativo:** Claves de tipo texto.
  `$u = ["user" => "Ana", "rol" => "Admin"];`

Funciones Frecuentes

- `count($a)` : Total de elementos.
- `in_array($val, $a)` : Busca si existe un valor.
- `sort()` / `asort()` / `ksort()` : Ordenaciones.
- `array_push()` / `array_pop()` : Pilas.

6. FORMULARIOS Y MANEJO DE FICHEROS

Peticiones y Subida (FILES)

- `GET` : Parámetros visibles en URL.
- `POST` : Parámetros ocultos en cuerpo HTTP.
- Subida de ficheros requiere:
  `enctype="multipart/form-data"` .
- `move_uploaded_file($temp, $destino)` .
- **Seguridad:** Usa `htmlspecialchars()` contra XSS.

Operaciones con Archivos

- `fopen($path, $modo)` : Abre descriptor.
- Modos: `'r'` (lectura), `'w'` (escritura/trunca), `'a'` (anexar al final).
- `fgets()` : Lee línea a línea.
- `unlink($path)` : Elimina un archivo.
- `header('Location: ...')` : Redirección.

7. PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

Resumen POO: Encapsulación con `public` , `private` y `protected` . Método `__construct()` para instanciar, palabra reservada `extends` para herencia simple, e `implements` para múltiples interfaces.

```php
<?php
namespace App\Modelos;

interface Auditable {
    public function log(): void;
}

abstract class Persona {
    protected string $nombre;
    public function __construct($nombre) { $this->nombre = $nombre; }
    abstract public function getRol(): string;
}

class Usuario extends Persona implements Auditable {
    private static int $count = 0;

    public function __construct($nombre) {
        parent::__construct($nombre);
        self::$count++;
    }

    public function getRol(): string { return "Usuario: " . $this->nombre; }
    public function log(): void { echo "Logging " . $this->nombre; }
    public static function getCount(): int { return self::$count; }
}
?>
```

¡Puntos clave para no perder nota en el examen!

- No confundir `$this->atributo` (instancia) con `self::$atributo` (estático de clase).
- PHP **NO** soporta herencia múltiple de clases directa, se simula con **Interfaces**.
- Las funciones abstractas obligan a la clase hija a implementarlas con la misma firma.

8. EXPRESIONES REGULARES EN PHP (PCRE)

Función Principal y Delimitadores

Las expresiones regulares en PHP requieren delimitadores (habitualmente `/`, o también `#`, `~`, `%`).

`preg_match($patron, $cadena, $coincidencias)`

- **Retorno:** Devuelve `1` (coincidencia), `0` (sin coincidencia) o `false` (error).
- **Estructura:** `"/patron/modificadores"` (ej. `"/[a-z]+/i"`).

Metacaracteres

`.`	Cualquier carácter
`\d`	Dígito numérico (0-9)
`\w`	Letra, número o guion bajo (_)
`\s`	Espacio, tabulación o salto de línea
`^` / `$`	Inicio / Fin de la cadena
`\b`	Límite de palabra exacta

Cuantificadores

`*`	0 o más veces
`+`	1 o más veces
`?`	0 o 1 vez (opcional)
`{n}`	Exactamente n veces
`{n,}`	Al menos n veces
`{n,m}`	Entre n y m veces

9. GRUPOS, RANGOS Y MODIFICADORES

Corchetes `[...]` y Paréntesis `(...)`

- `[aeiou]`: Coincide con una sola vocal.
- `[a-z]`, `[0-9]`: Rangos de caracteres.
- `[^0-9]`: Negación (NO es un número).
- `(abc)+`: Agrupa para aplicar repetición a la secuencia completa.
- `rojo|azul`: Alternancia (busca "rojo" O "azul").

Modificadores Principales

- `i`: Insensible a mayúsculas/minúsculas.
- `m`: Multilínea (trata saltos de línea como inicio/fin).
- `s`: Permite que el punto (`.`) incluya saltos de línea.

10. EXPRESIONES REGULARES EN LARAVEL

Reglas de Validación

Laravel utiliza internamente `preg_match`. La regla debe incluir delimitadores válidos:

```
$request->validate([
    'email' => 'required|regex:/^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$/i',
    // Si la regex contiene '|', usar formato de array:
    'codigo' => ['required', 'regex:/^[A-Z]{3}[0-9]{4}$/']
]);
```

¡Atención en Laravel!
Si tu expresión regular contiene la barra vertical `|` (operador "O"), debes definir las reglas de validación en un **array** en lugar de separarlas por cadenas con `|` para evitar que Laravel interprete mal las reglas.